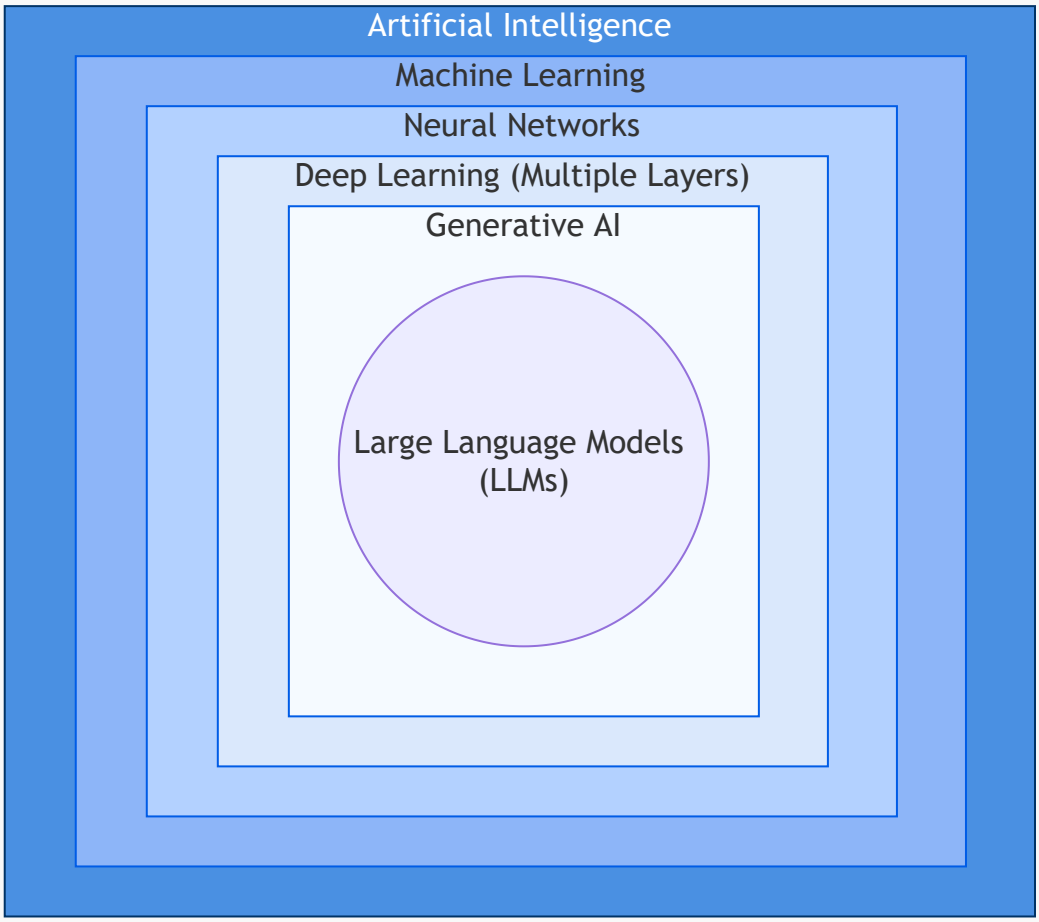
# AI outline graph




# Definition: Generative AI

讓computer學會產生複雜而有結構的物件

- 複雜 : 無限可能
- 有結構 : 由有限的基本單位(toekn)所構成

就是Classification, 因為Generative AI就是給一個未完成的句子, 去猜接下來接哪一個token

# Retrieval Augmented Generation(RAG)

# Context Engineering的核心目標

- 避免塞爆Context(把需要的放進去,不需要的清出來)
- 常用招數
  - Select
    - 挑選需要的內容, e.g. RAG, Tool RAG, Memory RAG
  - Compress
  - Multi-Agent

---

# Machine Learning

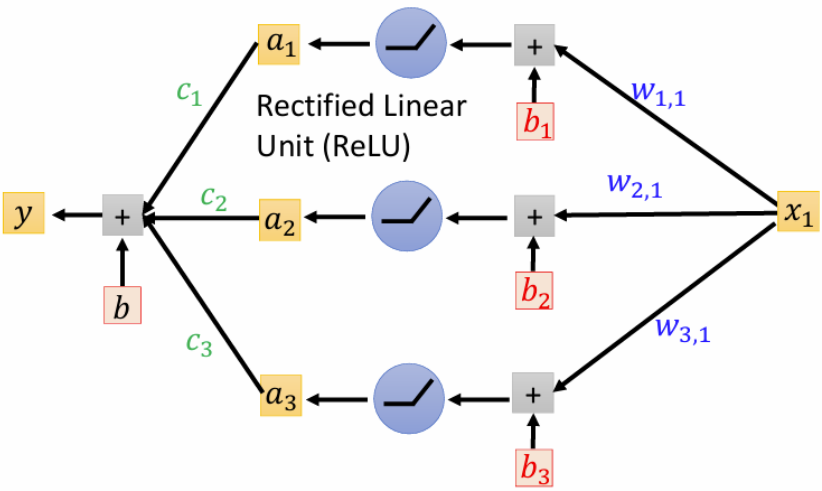
## 1. prepare data

- split to train data & test data

## 2. set a model

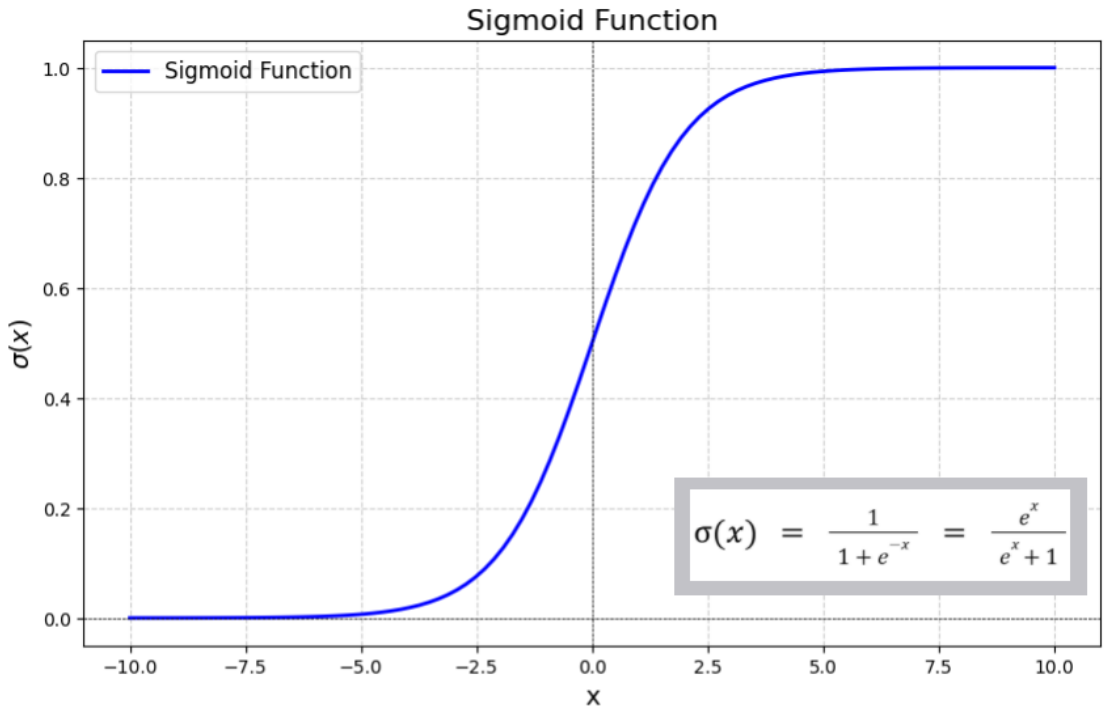
- 根據domain knowledge
  - Simple Linear Regression
    - $y = \beta_0 + \beta_1 x + \epsilon$
      - 其中 $\beta_0$ 為截距，$\beta_1$ 為斜率，$\epsilon$ 為隨機誤差。
  - Multiple Linear Regression
    - $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$
  - Piecewise Linear Curves

$$y = b + \sum_{i=1}^{H} c_i \underbrace{max(0, w_{i,1} x_1 + b_i)}_{a_i}$$




  - Logistic Regression(分類問題, classification)

- $P(y=1|x) = \frac{1}{1+e^{-(\beta_0+\beta_1 x_1+\cdots+\beta_n x_n)}}$




## 3. set Cost function/Loss function

- Mean Squared Error, MSE
  - $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
    - $n$ : 訓練資料的總數 (Train Data Size / Batch Size).
    - $y_i$ : 第 $i$ 筆資料的真實標籤 (Ground Truth).
    - $\hat{y}_i$ : 模型對第 $i$ 筆資料的預測值 (Prediction)。

## 4. set optimizer

- Gradient Descent(Vanilla Gradient Descent)
  - $\theta^{t+1} = \theta^t - \eta \cdot \nabla L(\theta^t)$
    - $\nabla L(w)$ 代表損失函數 $L$ 在當前參數 $w$ 位置的「斜率」或「坡度」
    - $\eta$ : The Learning Rate
  - 問題 : 只根據當下算出來的$g^t$來決定方向
    - 解法 : Gradient Descent + Optimizer
      - 根據$g^0, g^1, g^2, \ldots, g^t$ 一起來決定方向(調整learning rate)
        - Adagrad
          - for each dimension i : $\eta = \frac{\eta}{\sigma_i^t}$ , $\sigma_i^t = \sqrt{\sum_{i=0}^{t}(g_i^t)^2}$
          - 問題 : 每一個維度權重相同
        - RMSProp
          - 解決Adagrad : 最近算出來的gradient給比較大的影響
          - use $\alpha$
  - 可以用 Momentum (下滑之物體會有動量繼續前行, 不會直接殺停)來繼續找 global minimum用以脫離saddle point or local minimum
    - for each dimension i : $\theta^{t+1} = \theta^t - \eta \cdot m_i^t$ , $m_i^t$ = $g_i^0$ + $g_i^1$ + $g_i^2$ +...+ $g^t$

- Adam : RMSProp + Momentum
  - Compute gradient $\boldsymbol{g}^t = \nabla L(\boldsymbol{\theta}^t)$

**Momentum**

For each dimension $i$: $m_i^t = \beta m_i^{t-1} + (1-\beta) g_i^t$ $\qquad \theta_i^{t+1} \leftarrow \theta_i^t - \eta m_i^t$

**RMSProp**

For each dimension $i$: $\sigma_i^t = \sqrt{\alpha\left(\sigma_i^{t-1}\right)^2 + (1-\alpha)\left(g_i^t\right)^2}$ $\qquad \theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$

**Adam**

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} m_i^t$$

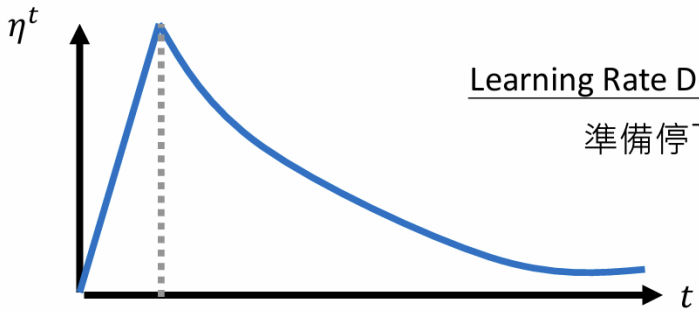
Adam has bias-corrected terms, which are omitted here for simplicity.

- 也可以用Learning Rate Scheduling來調整learning rate

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} m_i^t \quad \Rightarrow \quad \theta_i^{t+1} \leftarrow \theta_i^t - \frac{\boxed{\eta^t}}{\sigma_i^t} m_i^t$$

Warm Up

探索地形

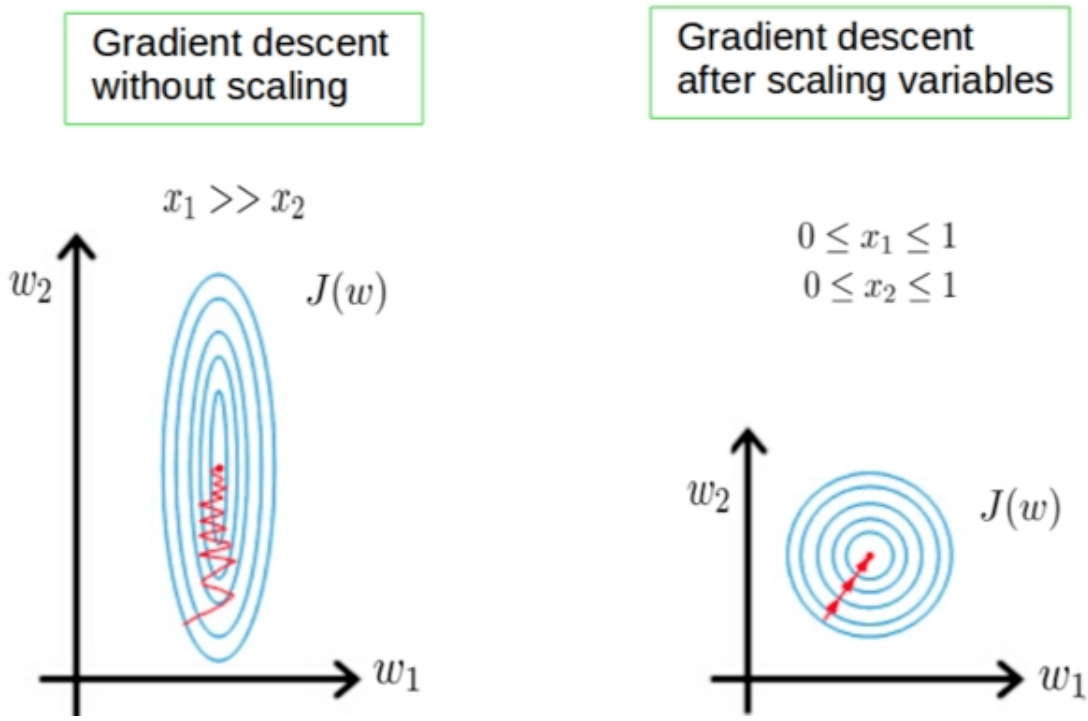



Learning Rate Decay

準備停下來

  - warp up : 給optimizer探索地形的機會, 因為剛進入一個新地圖, 不知道地圖有什麼, 設定一個大的learning rate, 讓參數亂跑, 可以大概知道地圖長什麼樣
- Feature Scaling

- accelerate gradient descent




- Normalization
  - Batch Normalization
  - Layer Normalization
- Standardizatoin

## 5.Train the model

- Initialization
  - Kaiming Initialization
- **Use gradient descent to train the model, accelerating convergence to the minimum loss and yielding the optimal model(Find the best $\beta_0, \beta_1, \beta_2, \beta_3, \ldots, \epsilon$ ).**

## 6. use validation data to validate your model

- be aware of overfitting

## 7. use test data to test your model

- public test data
  - be aware of overfitting
- private test data